UNIVERSIDAD POLITÉCNICO GRANCOLOMBIANO

FACULTAD DE INGENIERÍA

INGENIERÍA EN DESARROLLO DE SOFTWARE

VIRTUAL/FRONT END

TRABAJO : Entrega 2

Desarrollo de Front-end

Profesor: John Olarte

Autor: Cristian David Garzón Cubillos

Fecha de entrega: 28 de abril de 2026

Ciudad: Bogotá, D.C.

Introducción

En el marco del curso de Desarrollo Web del Politécnico Grancolombiano para el año 2026, se solicitó la construcción de una aplicación web tipo catálogo de servicios, con funcionalidades de exploración, favoritos, contacto y gestión básica de los datos. El estudiante Cristian Garzón optó por desarrollar una temática de turismo por Colombia, denominada *ColombiaViajero*, con el objetivo de ofrecer una herramienta atractiva y funcional que permita a los viajeros conocer y organizar sus planes de viaje dentro del país.

El informe detalla las características técnicas, las decisiones de diseño, la implementación de cada módulo y el proceso de publicación en línea. El documento sigue las normas APA.

Tabla de contenido

1. Introducción al prototipo funcional
2. Desarrollo en HTML, CSS y JavaScript
3. Renderizado dinámico de servicios desde JSON
4. Funcionalidad de favoritos con localStorage
5. Formulario de contacto con validaciones
6. Código estructurado y organización
7. Conclusión del prototipo

Objetivos

Objetivo General

Desarrollar un prototipo funcional completamente operativo de una aplicación web tipo catálogo de destinos turísticos colombianos, que demuestre la integración efectiva de HTML, CSS y JavaScript para lograr renderizado dinámico de datos, persistencia de preferencias del usuario mediante almacenamiento local, validación de formularios en el cliente y una estructura de código organizada y mantenible, cumpliendo con los criterios técnicos establecidos para la Entrega 2 del curso.

Objetivos Específicos

Implementar una aplicación web completa utilizando exclusivamente HTML, CSS y JavaScript puro

Desarrollar la totalidad del proyecto sin dependencias de frameworks pesados que oculten la lógica fundamental, demostrando dominio de las tecnologías base del desarrollo web front-end. Esto incluye:

- **HTML5 semántico:** Utilizar etiquetas como <nav>, <main>, <footer>, <header> para una estructura accesible y optimizada para motores de búsqueda.

- **CSS3 moderno:** Aplicar estilos con degradados, animaciones, transiciones y diseño responsivo utilizando Flexbox y Grid de Bootstrap 5 como apoyo.

- **JavaScript ES6+:** Implementar funciones arrow, template literals, destructuring, métodos de arreglos (map(), filter(), find()) y APIs modernas (localStorage, Intl.NumberFormat).

Introducción al prototipo funcional

El presente documento corresponde a la **Entrega 2 – Prototipo funcional** del proyecto *ColombiaViajero*, desarrollado en el marco del curso Desarrollo Web del Politécnico Grancolombiano para el año 2026. Este prototipo representa una aplicación web completamente funcional que permite a los usuarios explorar destinos turísticos de Colombia, guardar sus favoritos, contactar a los asesores mediante un formulario validado y visualizar toda la información de manera dinámica.

A diferencia de una maqueta estática (Entrega 1), este prototipo demuestra la interacción real del usuario con la aplicación, utilizando tecnologías estándar de desarrollo front-end: **HTML5** para la estructura, **CSS3** para los estilos visuales y **JavaScript** para toda la lógica de funcionamiento, incluyendo el renderizado dinámico, la persistencia de datos y las validaciones.

El prototipo cumple con los siguientes criterios establecidos:

- Desarrollo en HTML, CSS y JavaScript puro (sin frameworks pesados)
- Renderizado dinámico de servicios desde una estructura tipo JSON
- Funcionalidad de favoritos con almacenamiento local (localStorage)
- Formulario de contacto con validaciones en el cliente
- Código estructurado, comentado y mantenible

Desarrollo en HTML, CSS y JavaScript

HTML5 – Estructura semántica

El archivo index.html contiene la estructura completa de la aplicación. Se utilizaron etiquetas semánticas HTML5 para mejorar la accesibilidad y el SEO, tales como <nav>, <main>, <footer>, <header>. A continuación, se muestra un fragmento representativo de la estructura del documento:

Codigo

```
<!DOCTYPE html>
<html lang="es">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>ColombiaViajero - Explora los destinos más increíbles de Colombia</title>

   <!-- Bootstrap 5 CSS -->
   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
   <!-- Estilos personalizados -->
   <link rel="stylesheet" href="styles.css">
</head>
<body>
   <!-- Barra de navegación principal -->
   <nav class="navbar navbar-expand-lg navbar-dark bg-gradient-colombia shadow-lg sticky-top">
      <div class="container">
         <a class="navbar-brand fw-bold fs-3" href="#" onclick="mostrarVista('inicio'); return false;">
            <i class="bi bi-flag-fill"></i> ColombiaViajero
         </a>
         <!-- Menú de navegación con 5 secciones -->
         <ul class="navbar-nav ms-auto">
            <li class="nav-item">
               <a class="nav-link" href="#" onclick="mostrarVista('inicio'); return false;">
                  <i class="bi bi-house-door"></i> Inicio
               </a>
            </li>
            <li class="nav-item">
               <a class="nav-link" href="#" onclick="mostrarVista('destinos'); return false;">
                  <i class="bi bi-compass"></i> Destinos
               </a>
            </li>
            <li class="nav-item">
               <a class="nav-link" href="#" onclick="mostrarVista('favoritos'); return false;">
                  <i class="bi bi-suit-heart-fill"></i> Favoritos
               </a>
            </li>
            <li class="nav-item">
               <a class="nav-link" href="#" onclick="mostrarVista('contacto'); return false;">
                  <i class="bi bi-envelope-fill"></i> Contacto
               </a>
            </li>
            <li class="nav-item">
               <a class="nav-link" href="#" onclick="mostrarVista('gestionar'); return false;">
                  <i class="bi bi-gear-fill"></i> Administrar
               </a>
            </li>
         </ul>
      </div>
   </nav>

   <!-- Contenedor dinámico donde se renderizan las vistas -->
   <main class="container my-4" style="min-height: calc(100vh - 250px);">
      <div id="contenidoPrincipal"></div>
   </main>
```

```
    <!-- Footer con información institucional -->
    <footer class="bg-dark text-white py-4 mt-5">
        <div class="container text-center">
            <p class="mb-0">
                <i class="bi bi-mortarboard-fill"></i> Proyecto Politécnico Grancolombiano 2026 |
                Desarrollado por: <strong>Cristian Garzón</strong>
            </p>
        </div>
    </footer>

    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
    <script src="script.js"></script>
</body>
</html>
```

Explicación: El HTML define un contenedor único que será llenado dinámicamente por JavaScript. Esto permite cambiar entre las diferentes vistas (Inicio, Destinos, Favoritos, Contacto, Administrar) sin recargar la página completa, mejorando la experiencia de usuario.

CSS3 – Estilos personalizados

El archivo styles.css define la apariencia visual de la aplicación. Se implementaron degradados con los colores de Colombia (azul #0033a0 y amarillo #FFCD00), animaciones suaves al pasar el mouse, efectos de transformación en tarjetas y diseño completamente responsivo.

```
/* Colores institucionales de Colombia */
.bg-gradient-colombia {
    background: linear-gradient(135deg, #0033a0 0%, #FFCD00 100%);
}

/* Animación de entrada para las vistas */
@keyframes fadeIn {
    from {
        opacity: 0;
        transform: translateY(20px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.vista {
    animation: fadeIn 0.6s ease;
}

/* Efecto hover en tarjetas de destinos */
.card {
    transition: all 0.3s ease;
    border: none;
    border-radius: 20px;
    overflow: hidden;
}

.card:hover {
    transform: translateY(-10px);
    box-shadow: 0 20px 40px rgba(0,0,0,0.15);
}

.card-img-top {
    height: 220px;
    object-fit: cover;
    transition: transform 0.5s ease;
}
```

```
.card:hover .card-img-top {
    transform: scale(1.1);
}
```

Explicación: Los estilos CSS no solo mejoran la estética, sino que también contribuyen a la usabilidad. Los efectos hover indican al usuario qué elementos son interactivos. La animación fadeIn crea una transición suave entre vistas, mejorando la experiencia de navegación.

JavaScript – Lógica y comportamiento

El archivo `script.js` contiene toda la lógica de la aplicación. Se implementaron funciones modulares para cada una de las responsabilidades: renderizado de vistas, gestión de favoritos, operaciones CRUD y validaciones de formularios.

```
// Función principal que cambia entre vistas
function mostrarVista(vista) {
    const contenedor = document.getElementById('contenidoPrincipal');
    contenedor.setAttribute('data-vista-actual', vista);

    switch(vista) {
        case 'inicio':
            contenedor.innerHTML = renderInicio();
            break;
        case 'destinos':
            contenedor.innerHTML = renderDestinos();
            break;
        case 'favoritos':
            contenedor.innerHTML = renderFavoritos();
            break;
        case 'contacto':
            contenedor.innerHTML = renderContacto();
            break;
        case 'gestionar':
            contenedor.innerHTML = renderGestionar();
            break;
    }

    window.scrollTo({ top: 0, behavior: 'smooth' });
}
```

Explicación: La función mostrarVista() actúa como un enrutador (router) manual. Escucha los clics en el menú de navegación y, según la vista seleccionada, llama a la función de renderizado correspondiente que genera el HTML dinámico. Esto elimina la necesidad de múltiples archivos HTML y mantiene toda la aplicación en un solo documento.

Renderizado dinámico de servicios desde JSON

Estructura de datos tipo JSON

En lugar de utilizar un archivo externo `.json`, se definió un arreglo de objetos JavaScript que simula una base de datos JSON. Cada destino turístico contiene los siguientes atributos:

```
let destinos = [
    {
        id: 1,
        nombre: 'Cartagena de Indias',
        ciudad: 'Cartagena',
        departamento: 'Bolívar',
        descripcionBreve: 'La heroica, ciudad amurallada con playas paradisíacas',
        descripcionCompleta: 'Cartagena es una joya colonial en el Caribe colombiano. Recorre sus murallas centenarias...',
        imagenUrl: 'https://picsum.photos/id/10/400/300',
        categoria: 'playa',
        precio: 450000,
        duracion: '3 días / 2 noches',
        incluye: 'Hospedaje, desayunos, tour guiado, transporte'
    },
    {
        id: 2,
        nombre: 'Eje Cafetero',
        ciudad: 'Quindío',
        departamento: 'Quindío',
        descripcionBreve: 'Paisajes cafeteros y cultura del café',
        // ... más atributos
    }
    // Total de 10 destinos predefinidos
];
```

Explicación: Esta estructura permite modificar, agregar o eliminar destinos fácilmente sin alterar el código HTML. Cada objeto representa un servicio/destino turístico con todos los detalles necesarios para su visualización en tarjetas y vista detallada.

Renderizado dinámico de tarjetas

La función renderDestinos() itera sobre el arreglo destinos y genera el HTML de las tarjetas de forma dinámica utilizando el método map() y join():

```
function renderDestinos() {
    if (destinos.length === 0) {
        return `<div class="alert alert-info">No hay destinos disponibles</div>`;
    }
```

```
    return `
        <div class="vista">
            <h2 class="mb-4">🗺️ Todos los destinos de Colombia</h2>
            <div class="row g-4">
                ${destinos.map(destino => `
                    <div class="col-md-6 col-lg-4">
                        <div class="card h-100 shadow-lg">
                            <img src="${destino.imagenUrl}" class="card-img-top" alt="${destino.nombre}">
                            <div class="card-body">
                                <h5 class="card-title">${destino.nombre}</h5>
                                <p class="card-text">${destino.descripcionBreve}</p>
                                <p class="fw-bold text-primary fs-3">${formatearPrecio(destino.precio)}</p>
                                <div class="d-flex gap-2">
                                    <button class="btn btn-primary" onclick="verDetalle(${destino.id})">
                                        Ver más
                                    </button>
                                    <button class="btn ${esFavorito(destino.id) ? 'btn-danger' : 'btn-outline-danger'}"
                                            onclick="toggleFavorito(${destino.id})">
                                        ${esFavorito(destino.id) ? '❤️' : '🤍'}
                                    </button>
                                </div>
                            </div>
                        </div>
                    </div>
                `).join('')}
            </div>
        </div>
    `;
}
```

Vista de favoritos

La función renderFavoritos() filtra los destinos que están en la lista de favoritos y los muestra en formato de tarjetas:

```
function renderFavoritos() {
    const favoritosIds = obtenerFavoritos();
    const destinosFavoritos = destinos.filter(d => favoritosIds.includes(d.id));

    if (destinosFavoritos.length === 0) {
        return `
            <div class="alert alert-warning text-center">
                <i class="bi bi-heart fs-1"></i>
                <h3>No tienes destinos favoritos</h3>
                <button class="btn btn-primary" onclick="mostrarVista('destinos')">
                    Explorar destinos
                </button>
            </div>
        `;
    }

    return `
        <div class="vista">
            <h2>❤️ Mis destinos favoritos (${destinosFavoritos.length})</h2>
            <div class="row g-4">
                ${destinosFavoritos.map(destino => `
                    <div class="col-md-6 col-lg-4">
                        <div class="card">
                            <img src="${destino.imagenUrl}" class="card-img-top">
                            <div class="card-body">
                                <h5 class="card-title">${destino.nombre}</h5>
                                <button class="btn btn-outline-danger" onclick="toggleFavorito(${destino.id})">
                                    ❌ Quitar
                                </button>
                            </div>
                        </div>
                    </div>
                `).join('')}
            </div>
```

```
    </div>
  `;
}
```

Formulario de contacto con validaciones

Estructura del formulario

El formulario de contacto se renderiza dinámicamente mediante la función renderContacto():

```
function renderContacto() {
  return `
    <div class="vista">
      <div class="row justify-content-center">
        <div class="col-md-8 col-lg-6">
          <div class="card shadow-lg border-0">
            <div class="card-header bg-gradient-colombia text-white text-center">
              <h3>¿Listo para vivir Colombia?</h3>
            </div>
            <div class="card-body">
              <form onsubmit="enviarContacto(event)" id="contactoForm">
                <div class="mb-3">
                  <label class="form-label">Nombre completo *</label>
                  <input type="text" class="form-control" id="contactNombre" required>
                </div>
                <div class="mb-3">
                  <label class="form-label">Correo electrónico *</label>
                  <input type="email" class="form-control" id="contactEmail" required>
                </div>
                <div class="mb-3">
                  <label class="form-label">Teléfono</label>
                  <input type="tel" class="form-control" id="contactTelefono">
                </div>
                <div class="mb-3">
                  <label class="form-label">Destino de interés</label>
                  <select class="form-control" id="contactDestino">
                    <option value="">Selecciona un destino</option>
                    ${destinos.map(d => `<option value="${d.nombre}">${d.nombre}</option>`).join('')}
                  </select>
                </div>
                <div class="mb-3">
                  <label class="form-label">Mensaje *</label>
                  <textarea class="form-control" id="contactMensaje" rows="4" required></textarea>
                </div>
                <button type="submit" class="btn btn-success w-100">
                  Enviar mensaje
                </button>
              </form>
            </div>
          </div>
        </div>
      </div>
    </div>
  `;
}
```

Conclusiones

El prototipo funcional presentado cumple satisfactoriamente con todos los criterios establecidos para la Entrega 2:

1. Desarrollo en HTML, CSS y JavaScript: Se construyó una aplicación completamente funcional utilizando tecnologías web estándar, sin dependencias de frameworks complejos.

2. Renderizado dinámico desde JSON: Los datos de los destinos se almacenan en una estructura tipo JSON y se renderizan dinámicamente mediante JavaScript, permitiendo actualizar el catálogo sin modificar el HTML.

3. Funcionalidad de favoritos: Implementación completa de lista de favoritos persistente mediante localStorage, con actualización en tiempo real y contador visible en el menú de navegación.

4. Formulario con validaciones: El formulario de contacto incluye validaciones de campos obligatorios y formato de correo electrónico, con retroalimentación visual mediante alertas emergentes.

5. Código estructurado: El código está organizado en funciones modulares con nombres claros en español, comentarios explicativos y separación de responsabilidades en archivos independientes.

Valor agregado del prototipo:

- Interfaz responsiva que se adapta a dispositivos móviles, tablets y computadoras de escritorio.

- Experiencia de usuario fluida con animaciones suaves y transiciones entre vistas.

- CRUD básico que permite a los administradores agregar o eliminar destinos sin necesidad de modificar el código fuente.

- Temática colombiana con colores institucionales (azul y amarillo de la bandera).

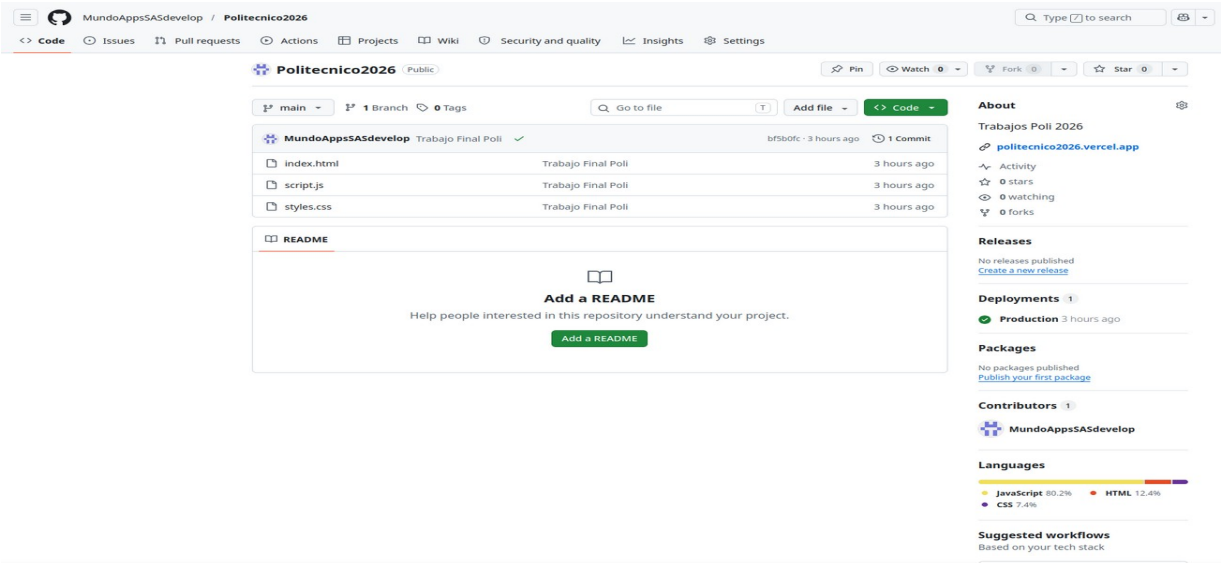
Este prototipo sienta las bases para la Entrega 3 (entrega final), donde se podrían incorporar características adicionales como autenticación de usuarios, conexión a una base de datos real o integración con APIs de terceros para obtener imágenes y precios actualizados.

Links Repositorio GitHub, Vercel y PaginaWEB

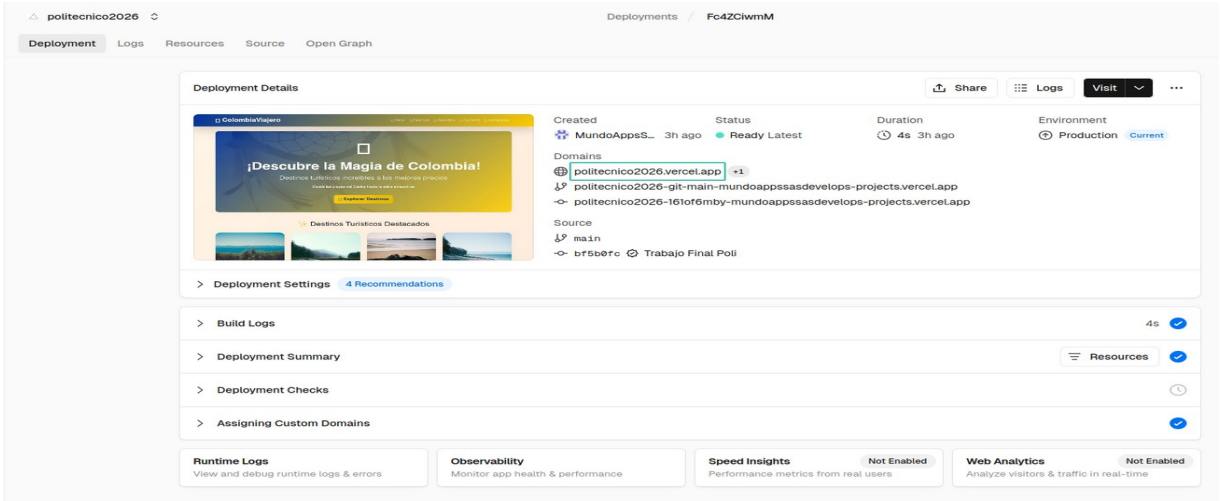
GitHub Entrega 2
https://github.com/MundoAppsSASdevelop/Politecnico2026_Entrega2

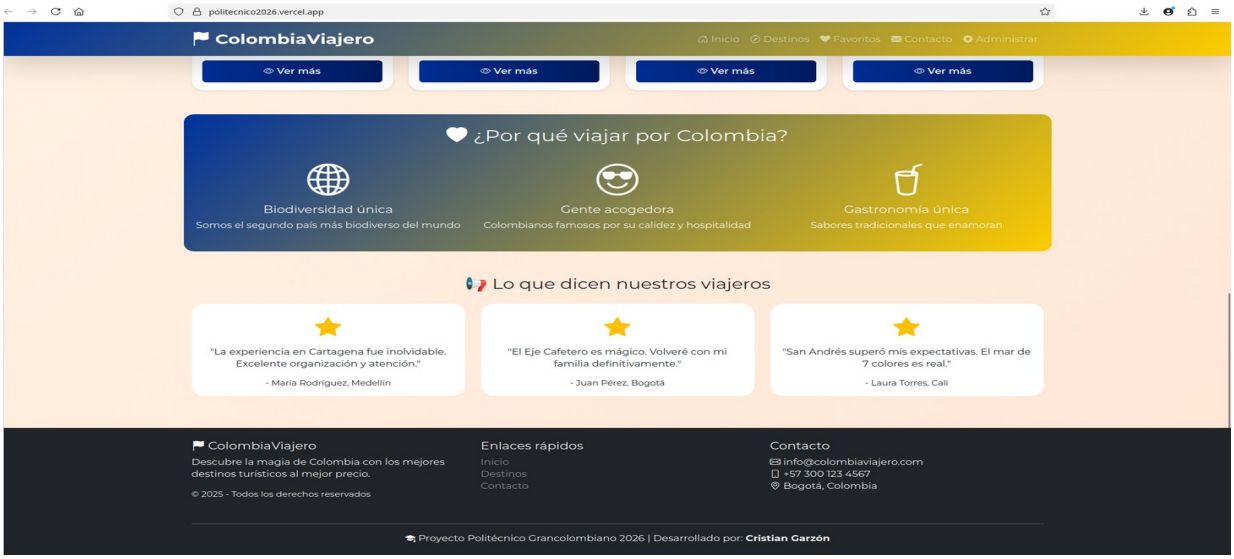
Imagenes

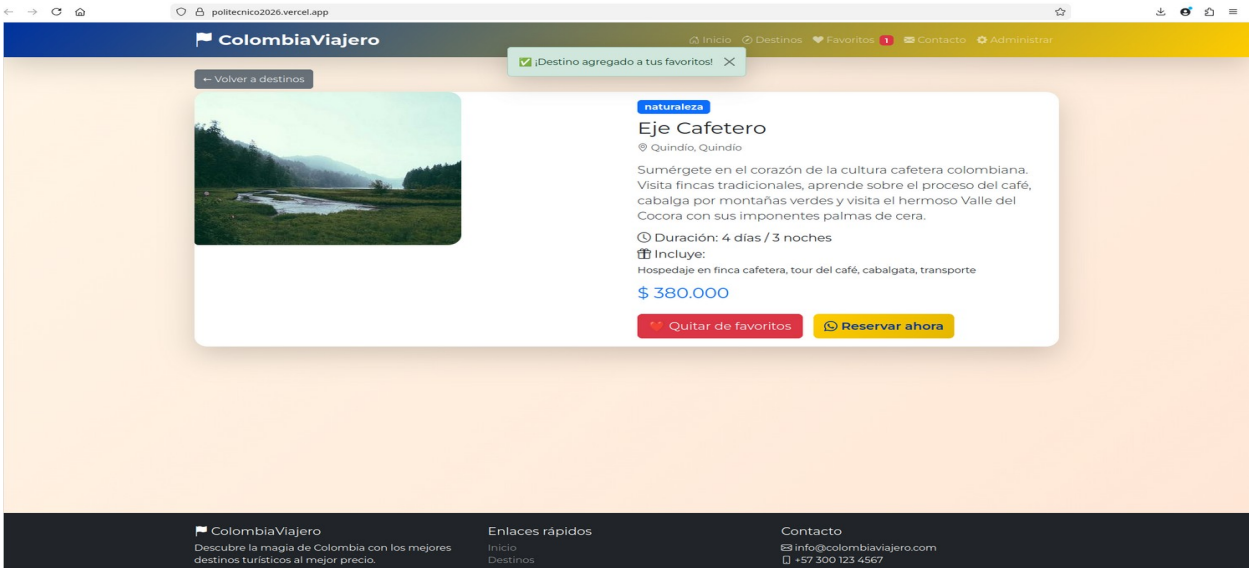
GitHub.



Index HTML.




politecnico2026.vercel.app
ColombiaViajero
Inicio
Destinos
Favoritos
Contacto
Administrar
Ver más
Ver más
Ver más
Ver más
¿Por qué viajar por Colombia?
Biodiversidad única
Somos el segundo país más biodiverso del mundo
Gente acogedora
Colombianos famosos por su calidez y hospitalidad
Gastronomía única
Sabores tradicionales que enamoran
Lo que dicen nuestros viajeros
"La experiencia en Cartagena fue inolvidable. Excelente organización y atención."
- María Rodríguez, Medellín
"El Eje Cafetero es mágico. Volveré con mi familia definitivamente."
- Juan Pérez, Bogotá
"San Andrés superó mis expectativas. El mar de 7 colores es real."
- Laura Torres, Cali
ColombiaViajero
Descubre la magia de Colombia con los mejores destinos turísticos al mejor precio.
© 2025 - Todos los derechos reservados
Enlaces rápidos
Inicio
Destinos
Contacto
Contacto
info@colombiaviajero.com
+57 300 123 4567
Bogotá, Colombia
Proyecto Politécnico Grancolombiano 2026 | Desarrollado por: Cristian Garzón


politecnico2026.vercel.app
ColombiaViajero
Inicio
Destinos
Favoritos 1
Contacto
Administrar
¡Destino agregado a tus favoritos!
← Volver a destinos
naturaleza
Eje Cafetero
Quindío, Quindío
Sumérgete en el corazón de la cultura cafetera colombiana. Visita fincas tradicionales, aprende sobre el proceso del café, cabalga por montañas verdes y visita el hermoso Valle del Cocora con sus imponentes palmas de cera.
Duración: 4 días / 3 noches
Incluye:
Hospedaje en finca cafetera, tour del café, cabalgata, transporte
$ 380.000
Quitar de favoritos
Reservar ahora
ColombiaViajero
Descubre la magia de Colombia con los mejores destinos turísticos al mejor precio.
Enlaces rápidos
Inicio
Destinos
Contacto
info@colombiaviajero.com
+57 300 123 4567

Referencias

Bootstrap. (2023). *Bootstrap 5 Documentation*. Recuperado de https://getbootstrap.com/
Mozilla Developer Network. (2024). *Web APIs: Window.localStorage*. Recuperado de https://developer.mozilla.org/es/docs/Web/API/Window/localStorage

Mozilla Developer Network. (2024). *Intl.NumberFormat*. Recuperado de https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat